

Static Members, Memory & Object Lifetime

Who owns the data? Where do references point? When can an object be reclaimed?

CSC241 - Object-Oriented Programming

Fall 2026 | COMSATS University Islamabad, Lahore Campus

Dr. Muhammad Shahid Bhatti

Assistant Professor | Department of Computer Science | Office H-17

Concept focus: ownership • references • lifetime • reachability

Student CLASS

static count = 3

object 1: Abeer Amina

object 2: Ali Ishtiaq

object 3: Roshan Ahmed

Many objects → one shared class-level member

Bridge from Lecture 6

Constructors initialize each object. Today we separate per-object state from class-wide state.

Lecture 6

Every new Student received its own id and name through a constructor.

New question

Should every Student object store another copy of a value that is common to all students?

Lecture 7

Use static for class-level members, then connect references and object lifetime to the JVM memory model.

new Student(...)

instance state

class-level state

lifetime

Theme: choose the correct owner of state — an individual object, the class as a whole, or a temporary method execution.

Lecture 7 scope from the course plan

Course-plan focus — Static Members & Memory Model

01

Static fields and methods vs. instance members

02

Stack frames vs. heap objects

03

Object lifecycle and garbage collection

04

Class initialization vs. object initialization

05

Class-level state and initialization order

06

Static constants and appropriate use of final

Core idea: choose the correct owner of data, then reason about how long that data remains reachable.

Learning outcomes

By the end of the lecture, you should be able to...

Classify members

as instance or static from their intended ownership.

Write static members

and call them using clear Java style.

Explain stack & heap

using a correct conceptual JVM model.

Trace reachability

and identify when an object becomes GC-eligible.

Trace initialization

class initialization once; object initialization per new.

Design constants

using static final where a value is shared and fixed.

Classroom habit: predict the output and draw the reference/memory picture before running the code.

Why do we need class-level members?

Some information belongs to every object separately; other information belongs to the class as a whole.

Per-object: instance state

Each student has a different id and name.
Each object must store its own values.

Shared: class-level state

How many Student objects have been created?
A single shared counter is enough.

INSTANCE FIELDS

```
private String id;  
private String name;
```

STATIC FIELD

```
private static int totalStudents;
```

Design question before syntax: “Who owns this value — one object or the class?”

Instance members — one copy per object

The receiver object determines which field values an instance method reads or changes.

INSTANCE STATE

```
public class Student {  
    private String id;  
    private String name;  
  
    public Student(String id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

Student object 1

id = ...005/LHR | name = Abeer Amina

Student object 2

id = ...010/LHR | name = Ali Ishtiaq

Student object 3

id = ...043/LHR | name = Roshan Ahmed

Changing object 1 does not automatically change the fields stored inside objects 2 and 3.

Static fields — one class-level variable

static changes the owner: from an individual object to the class.

ONE SHARED COUNTER

```
public class Student {  
    private String id;  
    private String name;  
  
    private static int totalStudents = 0;  
  
    public Student(String id, String name) {  
        this.id = id;  
        this.name = name;  
        totalStudents++;  
    }  
}
```

Instance fields

id and name belong to each Student object.

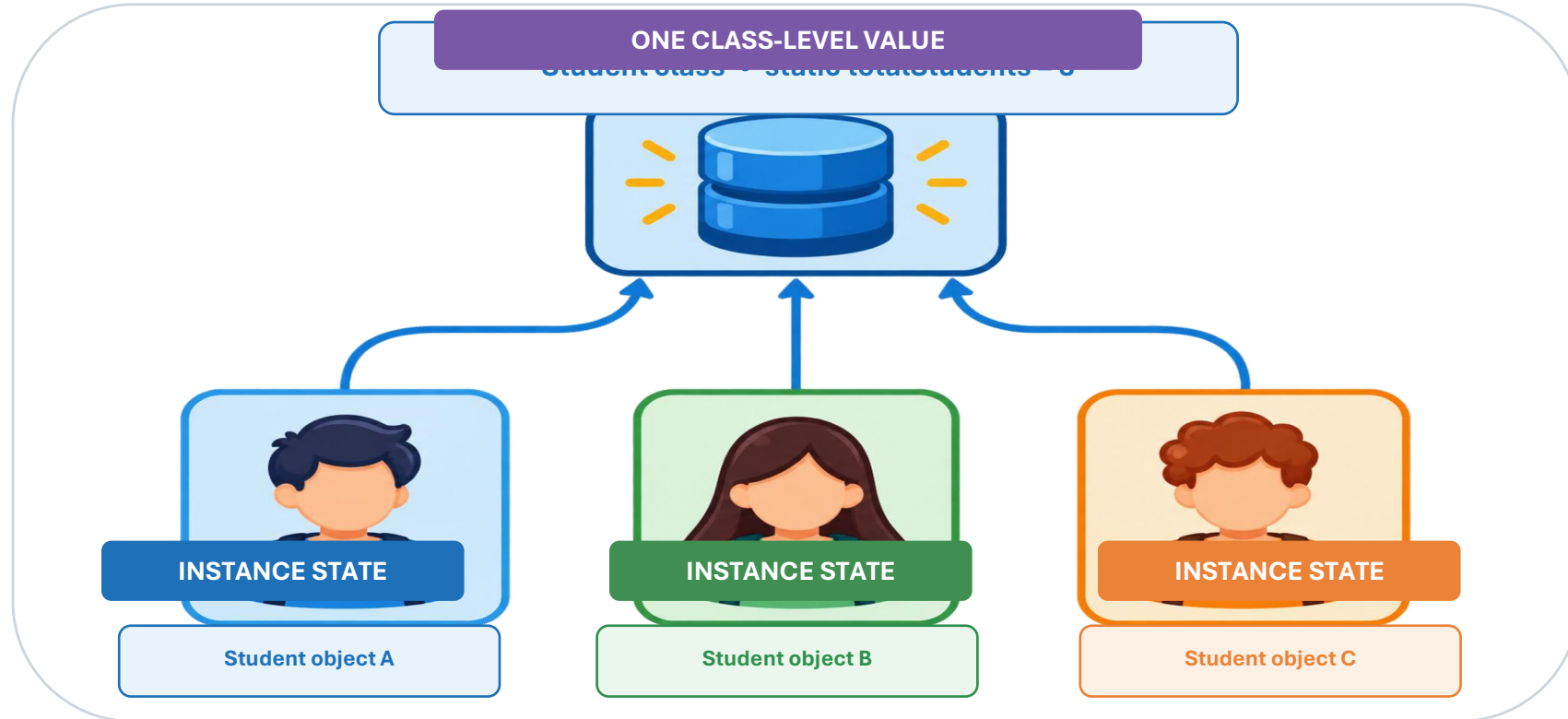
Static field

totalStudents belongs to the Student class and is shared.

Constructing any Student can update the same shared totalStudents variable.

One class, many objects, one static member

A memory-picture intuition for ownership.



Do not imagine three copies of totalStudents — there is one logical class-level value shared by all instances.

Live coding 1 — count constructed objects

Predict the counter after each new before compiling.

Student.java

```
public class Student {
    private String id;
    private String name;
    private static int totalStudents;

    public Student(String id, String name) {
        this.id = id;
        this.name = name;
        totalStudents++;
    }

    public static int getTotalStudents() {
        return totalStudents;
    }
}
```

Demo.java

```
public class Demo {
    public static void main(String[] args) {
        System.out.println(Student.getTotalStudents());
        new Student("...005/LHR", "Abeer Amina");
        System.out.println(Student.getTotalStudents());
        new Student("...010/LHR", "Ali Ishtiaq");
        System.out.println(Student.getTotalStudents());
    }
}
```

Predict

0 ? 1 ? 2 ?

Compile

javac *.java

Run

java Demo

Trace the shared counter

Every constructor call updates the same class-level variable.

Event	Student objects	totalStudents
Program starts	0	0
Construct Abeer	1	1
Construct Ali	2	2
Construct Roshan	3	3

Static does not mean “never changes.” It means “belongs to the class.” Mutability is a separate design choice.

Access static members through the class name

Use the class name for static members so the code visibly communicates ownership.

CALLING A STATIC METHOD

```
// clear and preferred
int n = Student.getTotalStudents();

// legal in some cases, but misleading
Student s = new Student("...", "...");
int m = s.getTotalStudents();
```

Preferred style

`Student.getTotalStudents()` says “this operation belongs to the class.”

Avoid misleading style

`s.getTotalStudents()` visually suggests that a particular `Student` object owns the value.

Style principle: write code that reveals the owner of the member without making the reader infer it.

Static methods — behavior that does not require a receiver object

A static method is invoked on the class and has no this reference.

STATIC UTILITY BEHAVIOR

```
public class IdTools {  
    public static boolean looksLikeLhrId(String id) {  
        return id != null && id.endsWith("/LHR");  
    }  
}  
  
boolean ok = IdTools.looksLikeLhrId(  
    "CIIT/SP26-BAI-005/LHR"  
);
```

No object required

You do not need new IdTools() merely to perform this operation.

No receiver

There is no particular IdTools object, therefore no this inside the static method.

Typical uses: utility operations, factories, counters, configuration/constants, and program entry points such as main.

Why can't a static method directly read an instance field?

Instance fields require a specific receiver object; a static call has no implicit receiver.

COMPILER ERROR

```
public class Student {  
    private String name;  
  
    public static void printName() {  
        System.out.println(name); // ERROR  
    }  
}
```

Student.printName()

Which object?

Abeer? Ali? Roshan? There is no this object to select one name field.

Fix: pass a Student reference as a parameter, or make the operation an instance method.

Live coding 2 — static utility vs. instance behavior

Let the class design reveal whether an object is actually needed.

IdTools.java

```
public final class IdTools {  
    private IdTools() { }  
  
    public static boolean isLhr(String id) {  
        return id != null && id.endsWith("/LHR");  
    }  
  
    public static String normalize(String id) {  
        return id == null ? "" : id.trim().toUpperCase();  
    }  
}
```

Demo.java

```
public class Demo {  
    public static void main(String[] args) {  
        String id = " ciit/sp26-bai-005/lhr ";  
        String clean = IdTools.normalize(id);  
        System.out.println(clean);  
        System.out.println(IdTools.isLhr(clean));  
    }  
}
```

Predict

What prints?

Compile

javac *.java

Explain

Why static?

The private constructor prevents meaningless IdTools objects.

Instance vs. static members — decision matrix

Ask who owns the state or behavior.

Question	Instance member	Static member
Owner	one object	the class
Copies	one logical value per object	one logical class-level value
Receiver / this	available	not available
Typical access	student.getName()	Student.getTotalStudents()
Can read instance fields directly?	yes, of its receiver	no
Good fit	object-specific state/behavior	shared state or receiver-free behavior

Rule of thumb: prefer instance members unless the concept genuinely belongs to the class or requires no receiver.

Static constants — shared and fixed class-level values

static chooses the owner; final prevents reassignment after initialization.

CLASS CONSTANTS

```
public final class TimeUnits {  
    public static final int SECONDS_PER_MINUTE = 60;  
    public static final int MINUTES_PER_HOUR = 60;  
  
    private TimeUnits() { }  
}
```

static

One shared value associated with the class.

final

The field cannot be assigned a different value after initialization.

Convention: class constants use UPPER_SNAKE_CASE names.

final is not the same as static

These keywords answer different design questions.

THREE DIFFERENT OWNERSHIP/MUTABILITY CHOICES

```
public class Student {  
    private final String id;           // per object, fixed  
    private String name;              // per object, mutable  
  
    public static final String CAMPUS = "LHR";  
                                     // class-level, fixed  
}
```

final only

One id field per Student object, but each object may hold a different id.

static only

One shared field; it may still be reassigned unless also final.

static final

One shared field and no reassignment after initialization — the common pattern for class constants.

Ownership and mutability are independent dimensions. static ≠ final.

A subtle point: final reference $\neq$ immutable object

final prevents the reference variable from pointing elsewhere; it does not automatically freeze the referenced object.

FINAL REFERENCE

```
public static final ArrayList<String> NAMES  
    = new ArrayList<>();  
  
NAMES.add("Abeer Amina");    // allowed  
NAMES.add("Ali Ishtiaq");    // allowed  
  
NAMES = new ArrayList<>();    // ERROR
```

What final protects

The NAMES variable cannot be reassigned to another ArrayList.

What final does not protect

The ArrayList object may still be mutable through its methods.

For true constants, prefer immutable values or expose mutable data very carefully.

Live coding 3 — constants + static method

Combine a fixed class rule with receiver-free behavior.

CreditMath.java — demonstration rules only

```
public final class CreditMath {  
    public static final int MIN_PER_COURSE = 1;  
    public static final int MAX_PER_COURSE = 6;  
  
    private CreditMath() { }  
  
    public static boolean validCredits(int credits) {  
        return credits >= MIN_PER_COURSE  
            && credits <= MAX_PER_COURSE;  
    }  
}
```

Demo.java

```
System.out.println(  
    CreditMath.validCredits(3)  
);  
  
System.out.println(  
    CreditMath.MAX_PER_COURSE  
);
```

Predict

Output?

Modify

Try 0, 1, 6, 7

Explain

Why static final?

These numeric limits are classroom demonstration values, not university policy.

Memory model for this lecture: stack, heap, class-level state

Use this as a reasoning model for Java programs — not as a literal map of physical RAM.

CALL STACK

- One frame per active method call
- Local primitives and local references
- A frame disappears when the method returns
- Each thread has its own stack

demo()

s = reference

HEAP

- Objects and arrays are allocated here
- References on the stack can point to them
- Shared across threads
- Unreachable objects become GC-eligible

Student object
id, name

CLASS-LEVEL STATE

- static fields logically belong to the class
- One shared class-level value
- Initialized during class initialization
- Lifetime is usually much longer than one method call

Student.count = 3

Draw references as arrows. A local reference variable is not the object itself.

Stack frames — temporary state of method execution

Each active method call has a frame. Local variables disappear with that frame; referenced objects do not necessarily disappear.

METHOD EXECUTION

```
static void demo() {  
    int x = 7;  
    Student s = new Student(  
        "...005/LHR",  
        "Abeer Amina"  
    );  
    s.printSummary();  
}
```

The local variable **s** is a reference value; it is not the **Student** object itself.

Current thread: JVM stack

printSummary() frame

receiver reference (this) + temporary locals

demo() frame

x = 7
s = reference to a Student object

FRAME ENDS ≠ OBJECT DESTROYED

The frame below waits while demo() executes.

Heap — where objects and arrays are reached through references

A heap object can outlive the method that created it whenever some reachable reference still points to it.

Shared JVM heap (conceptual)

Student object

id = ...005/LHR
name = Abeer Amina

Student object

id = ...010/LHR
name = Ali Ishtiaq

String object

"Abeer Amina"

int[] array

[2, 4, 6, 8]

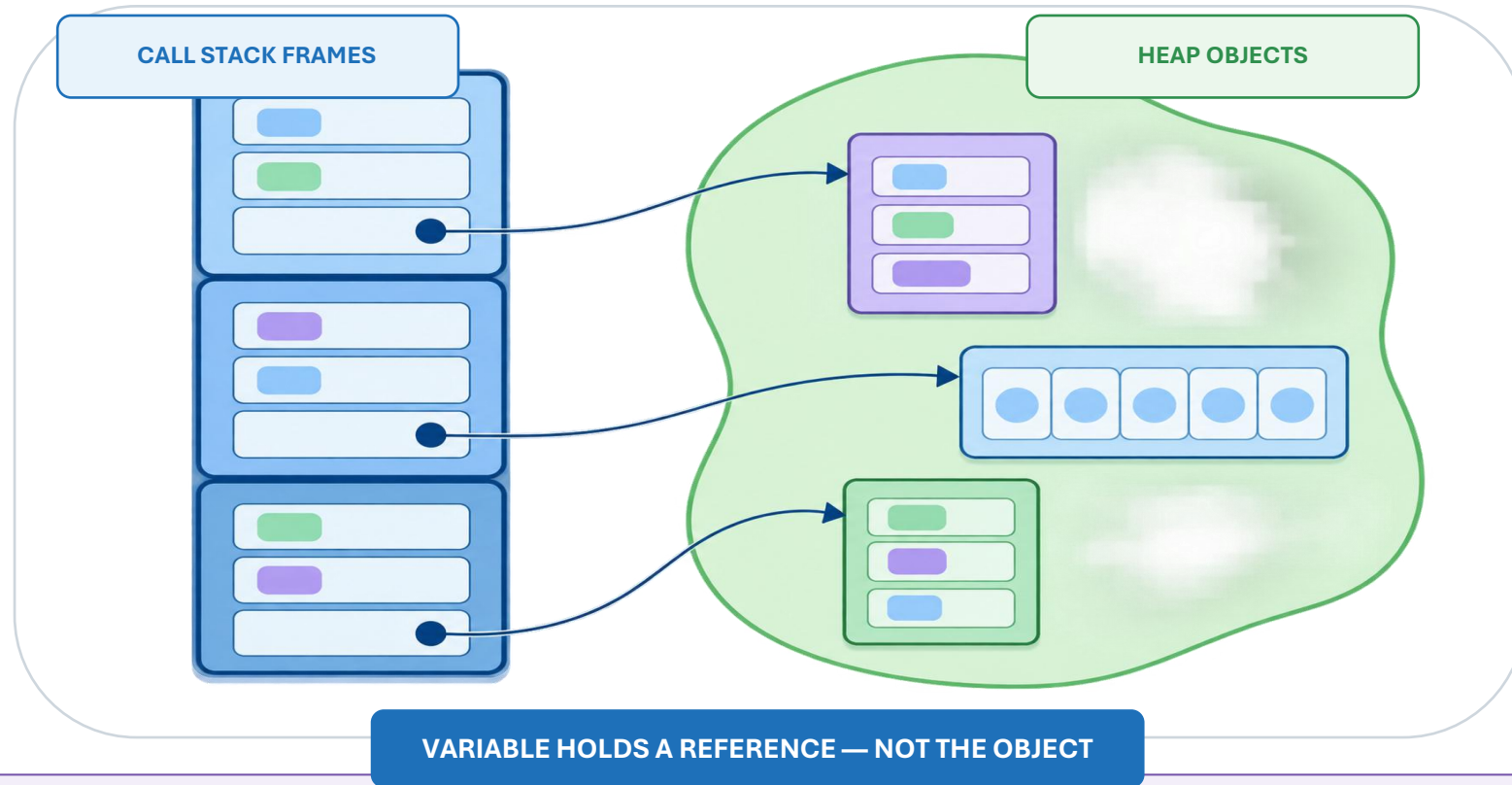
CourseSection object

code = CSC241

Heap storage is shared across threads; thread safety is a later concern when shared mutable state is involved.

Stack + heap together

Trace variables as arrows to objects rather than drawing the object “inside” the local variable.



When `demo()` returns, its frame disappears. The `Student` object remains only if some reachable reference still points to it.

Aliasing — multiple references can reach one object

An object stays reachable as long as at least one live reference path reaches it.

TWO REFERENCES, ONE OBJECT

```
Student a = new Student("...005/LHR", "Abeer Amina");  
Student b = a;
```

```
a = null;
```

```
// b still reaches the object  
System.out.println(b.getName());
```

a = null

b

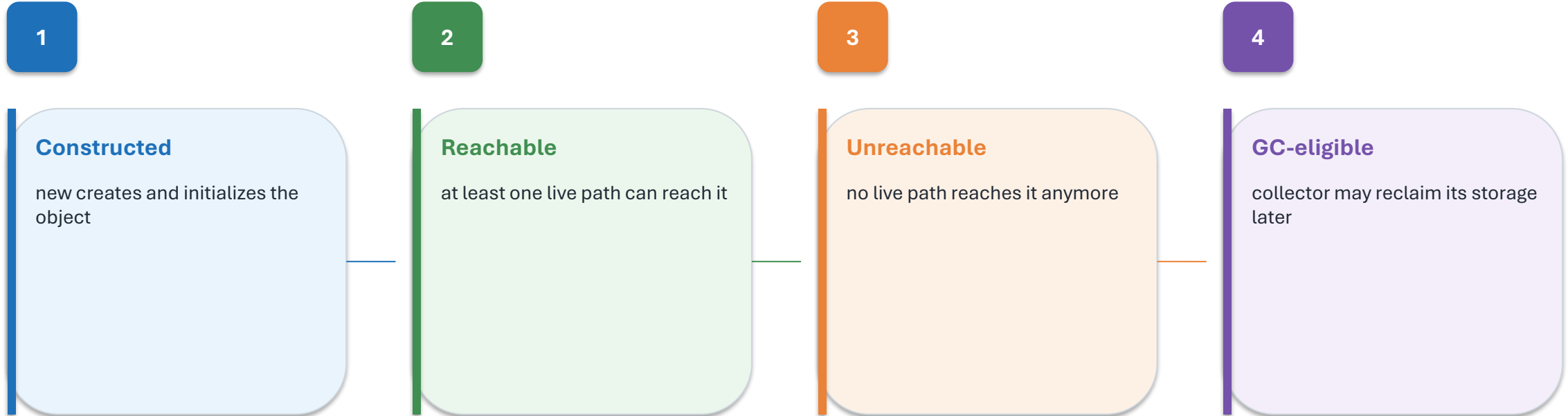
Student object

Abeer Amina
...005/LHR

Setting one reference to null does not destroy the object if another reachable reference still exists.

Object lifecycle — construction, reachability, GC eligibility

For ordinary Java objects, lifetime is governed by reachability — not by an explicit delete/free statement.



lifetime progresses by reachability—not by explicit delete/free

GC-eligible means “may be reclaimed later” — not “destroyed immediately.”

Garbage collection — reclaim memory by reachability

The collector identifies objects that are no longer reachable from live roots and may reclaim their storage.

1 Reachability matters

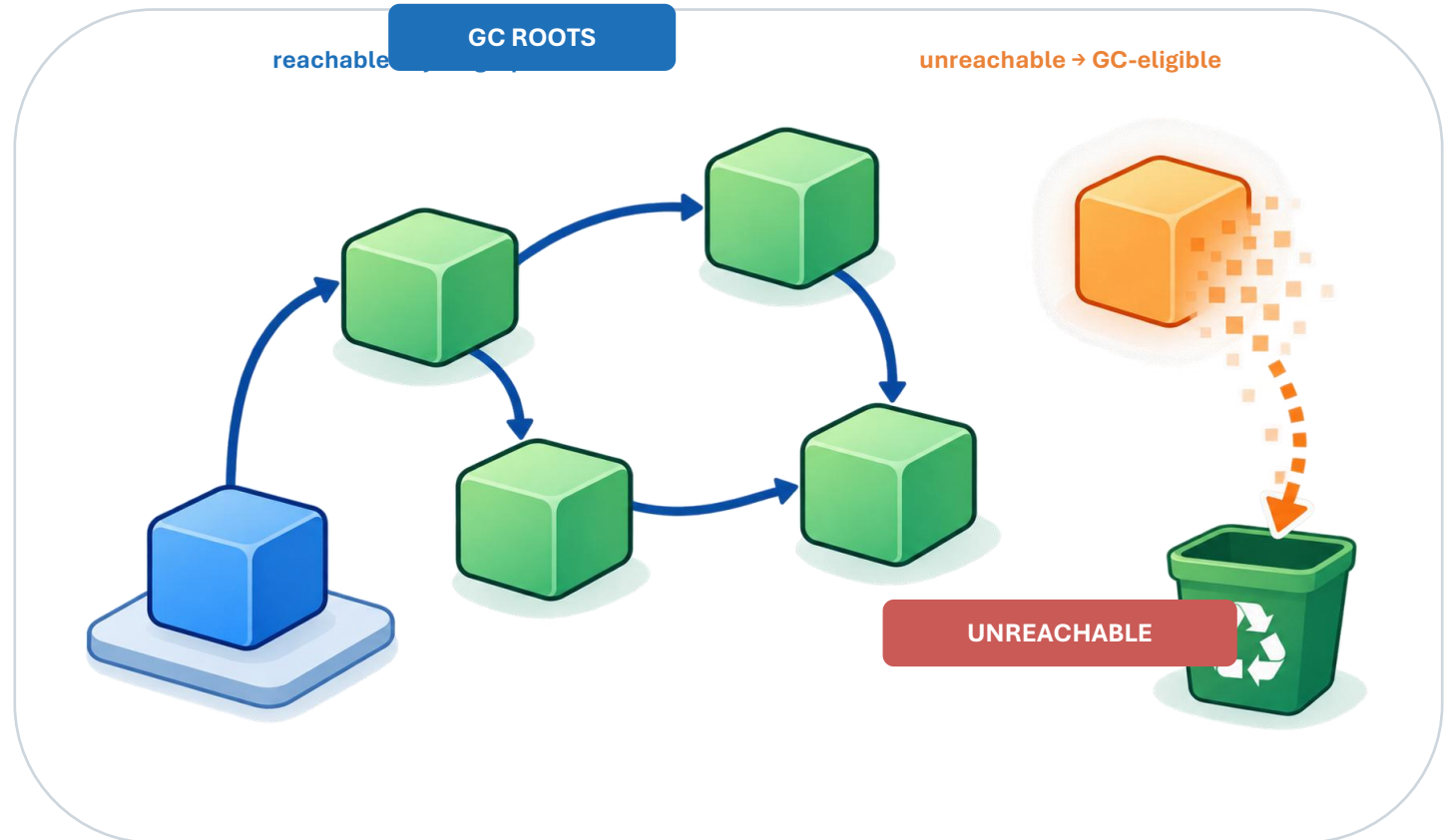
GC reasons about reachability from live roots—not merely whether one variable was set to null.

2 No timing guarantee

An unreachable object becomes eligible for collection, but reclamation may happen later.

3 No ordinary delete

Java code normally drops references and lets the JVM manage object memory.



`System.gc()` is only a request/hint; program correctness must never depend on exactly when collection occurs.

Prediction exercise — when does the object become unreachable?

Trace scope and references. Do not guess based on the last line that “looks like use.”

TRACE THIS

```
static Student make() {  
    Student x = new Student(  
        "...043/LHR", "Roshan Ahmed"  
    );  
    return x;  
}  
  
static void demo() {  
    Student s = make();  
    System.out.println(s.getName());  
    s = null;  
    // question: can the Student be reached now?  
}
```

At return from make()

The local x disappears, but its returned reference has been copied into s.

After s = null

Assuming no other reference was stored elsewhere, the object becomes unreachable.

Eligible ≠ reclaimed now. The GC chooses when to perform reclamation.

A GC language can still leak memory

A leak means objects remain reachable even though the application no longer needs them.

UNINTENDED RETENTION

```
public class StudentCache {  
    private static final ArrayList<Student> CACHE  
        = new ArrayList<>();  
  
    public static void remember(Student s) {  
        CACHE.add(s);  
    }  
}  
  
// Objects in CACHE remain strongly reachable  
// until removed (or the class/runtime is unloaded).
```

Why static matters

A long-lived static collection can itself remain reachable for a very long time.

GC is doing its job

It must not reclaim objects that are still reachable through CACHE.

Design lesson: shared mutable static state should be intentional and bounded.

Live coding 4 — reachability without depending on GC timing

Use references and scopes to reason; do not try to “prove” collection with `System.gc()`.

TRACE REFERENCES, NOT COLLECTION TIMING

```
public class ReachabilityDemo {
    static Student saved;

    static void create() {
        Student local = new Student(
            "...010/LHR", "Ali Ishtiaq"
        );
        saved = local;
    }

    public static void main(String[] args) {
        create();
        System.out.println(saved.getName());
        saved = null;
    }
}
```

Predict 1

After `create()` returns, is the `Student` reachable?

Predict 2

After `saved = null`, what reference path remains?

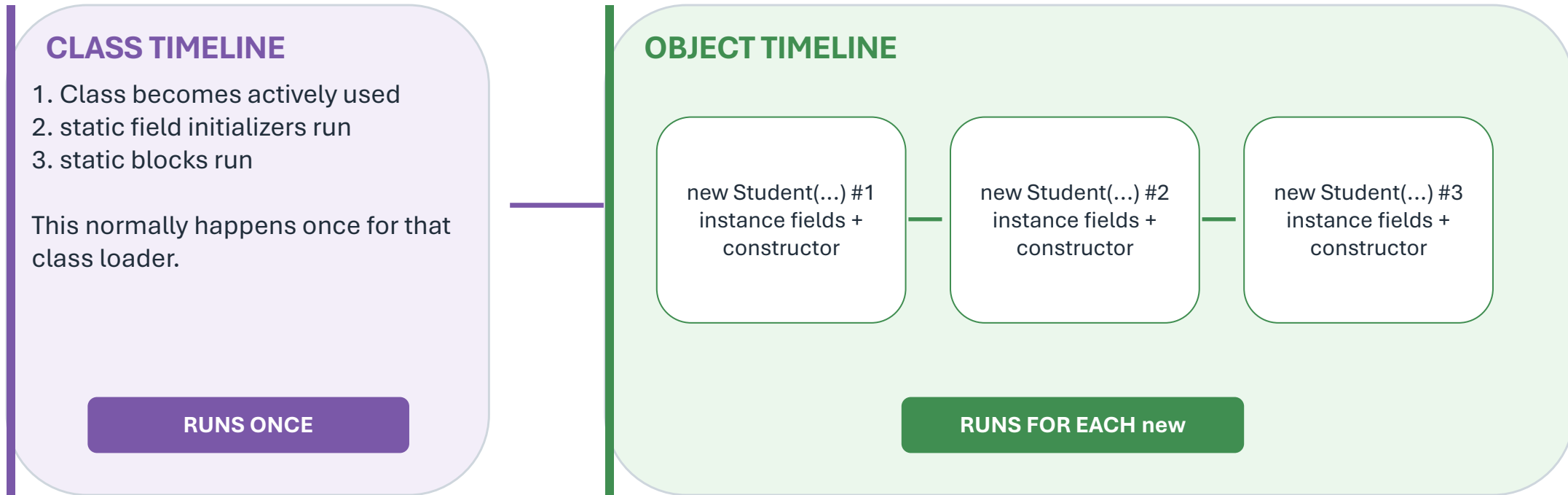
Explain

Why should we not print “object collected” as a deterministic outcome?

Goal: understand eligibility, not force the collector.

Class vs. object initialization — two timelines

The class is initialized once before active use; object initialization happens again for every new object.



Memory cue: static → class timeline; instance fields + constructor → object timeline.

When is static state created and initialized?

Separate storage/default preparation from execution of explicit static initialization code.

STATIC MEMBERS

```
public class Counter {  
    static int a;           // default: 0  
    static int b = 10;      // explicit initializer  
  
    static {  
        a = 5;  
        b++;  
    }  
}
```

During class preparation

Static storage is prepared and fields receive their initial/default setup according to JVM rules.

During class initialization

Explicit static field initializers and static initializer blocks execute in source order.

For this example after initialization: a = 5, b = 11.

Static initialization order — textual order matters

Static field initializers and static blocks are executed once, in their source order during class initialization.

SOURCE ORDER

```
public class Trace {  
    static int a = initA();  
  
    static {  
        System.out.println("static block");  
    }  
  
    static int b = 20;  
  
    static int initA() {  
        System.out.println("initA");  
        return 10;  
    }  
}
```

1

a initializer calls initA() → prints initA

2

static block executes → prints static block

3

b gets explicit value 20

Runs once for class initialization, not once per object.

Live coding 5 — trace class initialization vs. constructors

Predict the exact output order before compiling.

Student.java

```
public class Student {
    static int count = announceCount();

    static {
        System.out.println("Student class initialized");
    }

    Student() {
        count++;
        System.out.println("constructor: " + count);
    }

    static int announceCount() {
        System.out.println("count initialized");
        return 0;
    }
}
```

Demo.java

```
public class Demo {
    public static void main(String[] args) {
        System.out.println("A");
        new Student();
        new Student();
        System.out.println("B");
    }
}
```

Predict

Exact lines?

Compile

javac *.java

Explain

Which runs once?

Static initialization occurs before the first Student constructor.

When does class initialization happen?

For this course, think “first active use.” The JVM specification contains advanced edge cases that are not the main tracing target here.

`new Student()`

Creating the first Student object requires the Student class to be initialized first.

`Student.someStaticMethod()`

Invoking a static method can trigger initialization if the class is not already initialized.

Read/write static field

Access to a non-constant static field can trigger class initialization.

Optional nuance: a compile-time constant may be inlined into client bytecode, so reading it may not initialize the declaring class.

For normal CSC241 reasoning: class-level initialization happens once before the class is actively used; constructors then run per object.

Class initialization vs. object initialization

Two timelines — once per class, then once per object.

Aspect	Class initialization	Object initialization
Frequency	normally once per class loader	every successful new
Main targets	static fields / static blocks	instance fields / constructor
Receiver object	none	the newly allocated object
Example	<code>static int count = 0;</code>	<code>this.id = id;</code>
Use case	shared class setup	per-object valid starting state

Connection to Lecture 6: constructors belong to the per-object timeline, not the class-initialization timeline.

Optional deep dive — bytecode reflects instance vs. static access

Use this only if time allows. The goal is conceptual reinforcement, not bytecode memorization.

Example.java

```
class Example {  
    int x;  
    static int y;  
  
    int readX() { return x; }  
    static int readY() { return y; }  
}
```

COMMAND LINE

```
$ javac Example.java  
$ javap -c Example  
  
... getfield    ... Example.x:I  
... getstatic  ... Example.y:I
```

getfield

Requires an object reference because x belongs to an instance.

getstatic

Refers to class-level state because y belongs to the class.

This is a conceptual reinforcement, not a bytecode-memorization requirement.

Integrated live challenge — Student registry counter

Use encapsulation + constructors + static + final without introducing later-course features.

COMPLETE THE TRACE

```
public class Student {  
    public static final String CAMPUS = "LHR";  
    private static int count;  
  
    private final String id;  
    private String name;  
  
    public Student(String id, String name) {  
        this.id = id;  
        this.name = name;  
        count++;  
    }  
  
    public static int getCount() { return count; }  
    public String getName() { return name; }  
}
```

CLIENT CODE

```
Student a = new Student("...005/LHR", "Abeer Amina");  
Student b = new Student("...043/LHR", "Roshan Ahmed");  
  
System.out.println(Student.CAMPUS);  
System.out.println(Student.getCount());  
System.out.println(a.getName());
```

Predict

3 output lines

Draw

stack + heap

Classify

static vs instance

Then add a third object and predict what changes — and what does not.

Rapid concept check — true or false?

Ask students to vote first, then reveal the reason.

“static means constant”

False — static controls ownership; final controls reassignment.

“static has one copy per object”

False — instance fields do; static is class-level.

“local reference = object”

False — the variable holds a reference value.

“null immediately destroys object”

False — reachability through all references matters.

“GC runs when I call System.gc()”

Not guaranteed — it is only a request/hint.

“static method can use this”

False — there is no receiver object for a static call.

Two questions solve most mistakes: “who owns it?” and “what reference path still reaches it?”

Exit ticket + transition to OO problem solving

Consolidate ownership, references, and lifetime before moving to class design.

Exit ticket — 3 minutes

1. Difference between static and instance fields?
2. When does an object become GC-eligible?
3. Why can't a static method use this?

Mini practice — think before coding

```
Student a = new Student(...);  
Student b = a;  
a = null;
```

1. Is the object still reachable?
2. Which reference keeps it alive?
3. Is Student.count instance or static?

Ready for Lecture 8 when...

You can separate object state from class-wide state.
You can draw references as arrows.
You can explain when an object becomes GC-eligible.

Lecture 6

Lecture 7

Lecture 8 — OO Problem Solving

Next: identify candidate classes, responsibilities, collaborators, and a first conceptual class diagram from a problem statement.